

Chapter 1

Congestion and Curtailment Benefits Calculation Documentation

1.1 Introduction

The `congestion_curtailment_reduction.py` script calculates the benefits from reducing transmission congestion and renewable energy curtailment. It allocates effective capacity relief between curtailment and congestion to avoid double-counting, then monetizes both benefits over the project lifetime.

1.2 Method Overview

1.2.1 Purpose

This script calculates congestion and curtailment benefits by:

1. Determining effective capacity relief (greenfield vs. reconductoring)
2. Allocating capacity relief between curtailment and congestion
3. Calculating congestion reduction energy and costs
4. Calculating curtailment reduction energy and benefits
5. Applying saturation factors (haircuts) to account for diminishing returns
6. Computing residual exceedance costs (unmet capacity needs; one cost from congestion and/or curtailment)
7. Calculating present values of all benefits and costs

1.2.2 Calculation Approach

The calculation follows this structure:

Effective Capacity Relief:

$$\Delta C_{eff} = \begin{cases} \phi \times \text{Capacity} & (\text{Greenfield}) \\ \text{Capacity} - \text{Old Capacity} & (\text{Reconductoring}) \end{cases} \quad (1.1)$$

where ϕ is the flow factor.

Curtailment Benefits:

$$E_{curt} = H_c \times \min(\Delta C_{eff}, C_c) \quad (1.2)$$

$$\text{Benefit}_{curt} = E_{curt} \times \lambda_c \quad (1.3)$$

where H_c is curtailment hours, C_c is average curtailment MW, and λ_c is curtailment price.

Congestion Benefits:

$$E_{cong} = E_{cong,bc} + E_{cong,non} + E_{near} \quad (1.4)$$

$$\text{Benefit}_{cong} = E_{cong} \times \lambda_{cong} \times (1 - \text{saturation}) \quad (1.5)$$

where the energy is split between overlap hours (with curtailment) and non-overlap binding hours.

Residual Exceedance: Residual exceedance is one cost that can have two sources—congestion binding hours and/or curtailment hours—so it works for congestion-only, curtailment-only, or both. When only one of congestion or curtailment is used, the other term is zero; when both are used, the user should set `residual_exceedance_value` if the single default (\$/MWh) is not appropriate.

$$E_{res,cong} = (H_{bc} + H_{bnon}) \times \max(0, X_{cong} - \Delta C_{eff}) \quad (1.6)$$

$$E_{res,curt} = H_c \times \max(0, C_c - \Delta C_{eff}) \quad (1.7)$$

$$E_{residual} = E_{res,cong} + E_{res,curt} \quad (1.8)$$

$$\text{Cost}_{residual} = E_{residual} \times \lambda, \quad \lambda = \text{residual_exceedance_value if set, else } \lambda_{cong} \quad (1.9)$$

1.2.3 Inputs and Outputs

Inputs:

- Project technical details (capacity, old capacity, reconductoring status)
- Congestion parameters (binding hours, average exceedance, flow factor, saturation factor, congestion price; `residual_exceedance_value` optional, null = use average congestion price)
- Curtailment parameters (curtailment hours, average curtailment MW, curtailment price, saturation factor)

- Financing parameters (WACC, timeline)

Outputs:

- Annual and lifetime congestion benefits (raw and haircut)
- Annual and lifetime curtailment benefits (raw and haircut)
- Residual exceedance costs
- Congestion and curtailment costs during delay/construction (allocated, no double count)
- Present values of all metrics

1.2.4 Integration with Other Scripts

This script integrates with:

- `smart_loaders.py` — `load_congestion_curtailment_reductions`, `load_project_technical_details`, `load_financing_details`, `get_project_data_raw`
- `yaml_loaders.py` — `ProjectTechnicalDetails` (via centralized loader)
- `financial_utils.py` — `calculate_present_value`, `calculate_cod_year`
- `constants.py` — `MIN_DISCOUNT_RATE`
- `smart_output.py` — `CTCCOutputManager` (`add_congestion_curtailment`, `write_batch_summary`)

1.3 Key Functions

1.3.1 `allocate_curtailment_then_congestion(DC_eff_mw, ...)`

Allocates effective capacity relief between curtailment and congestion to avoid double-counting.

Parameters:

- `DC_eff_mw` (float): Effective capacity relief in MW
- `binding_hours_total` (float): Total binding constraint hours
- `average_exceedance_mw` (float): Average exceedance in MW
- `curtailment_hours_total` (float): Total curtailment hours
- `average_curtailment_mw` (float): Average curtailment in MW
- `average_curtailment_price` (float): Average curtailment price

Returns:

- Dictionary with curtailment benefits, overlap metrics, and remaining capacity for congestion

Key Logic:

```

1 Hb = max(0.0, binding_hours_total)
2 Hc = max(0.0, curtailment_hours_total)
3 # Curtailment relief (MWh/yr)
4 E_curt = Hc * min(DeltaC_eff_mw, Cc)
5 curt_benefit = E_curt * lambda_c
6
7 if Hb <= 0:
8     return {..., "H_bc": 0, "H_bnon": 0, "DeltaC_remain_bc_mw":
9         DeltaC_eff_mw}
10
11 # Overlap proxy between binding and curtailment time
12 theta = min(1.0, Hc / Hb) if Hc > 0 else 0.0
13 H_bc = theta * Hb
14 H_bnon = (1.0 - theta) * Hb
15 DeltaC_used_bc = min(DeltaC_eff_mw, Cc)
16 DeltaC_remain_bc = max(0.0, DeltaC_eff_mw - DeltaC_used_bc)
17 # Returns E_curt_mwh_yr, curtailment_benefit_$_yr, theta_overlap,
18     H_bc, H_bnon, DeltaC_remain_bc_mw

```

1.3.2 calculate_congestion_reduction_costs(...)

Main function that calculates congestion and curtailment benefits.

Parameters:

- Multiple parameters including reconductoring status, capacities, congestion/curtailment parameters, financing parameters

Returns:

- Dictionary with all calculated benefits, costs, and present values

Key Logic:

```

1 if not reconductoring:
2     effective_capacity_relief = max(0, flow_factor * capacity_mw)
3 else:
4     effective_capacity_relief = max(0, capacity_mw - old_capacity_mw)
5
6 alloc = allocate_curtailment_then_congestion(DeltaC_eff_mw=
7     effective_capacity_relief, ...)
8 DeltaC_rem = alloc["DeltaC_remain_bc_mw"]
9 H_bc, H_bnon = alloc["H_bc"], alloc["H_bnon"]

```

```

10 E_cong_bc = H_bc * min(DeltaC_rem, average_exceedance)
11 E_cong_non = H_bnon * min(effective_capacity_relief,
    average_exceedance)
12 E_near = k * near_binding_hours * effective_capacity_relief
13 energy_congestion_reduction = E_cong_bc + E_cong_non + E_near
14
15 annual_congestion_reduction_cost_raw = energy_congestion_reduction *
    average_congestion_price
16 annual_congestion_reduction_cost_haircut = (1 - saturation_factor) *
    annual_congestion_reduction_cost_raw
17
18 total_binding_hours = H_bc + H_bnon
19 energy_residual_congestion = total_binding_hours * max(0,
    average_exceedance - effective_capacity_relief)
20 energy_residual_curtailement = curtailment_hours_total * max(0,
    average_curtailement_mw - effective_capacity_relief)
21 energy_residual_exceedance = energy_residual_congestion +
    energy_residual_curtailement
22 residual_exceedance_value_used = residual_exceedance_value or
    average_congestion_price
23 annual_residual_exceedance_cost = energy_residual_exceedance *
    residual_exceedance_value_used

```

1.3.3 main()

Main execution function that orchestrates the calculation and output.

1.4 Example

The following example demonstrates a typical usage:

```

1 # Example: 500 MW Greenfield Overhead AC line
2 #
3 # Inputs (from YAML):
4 #   - Capacity: 500 MW
5 #   - Flow factor: 0.7
6 #   - Binding hours: 2000 hours/year
7 #   - Average exceedance: 300 MW
8 #   - Curtailment hours: 1500 hours/year
9 #   - Average curtailment: 200 MW
10 #   - Congestion price: $50/MWh
11 #   - Curtailment price: $30/MWh
12 #   - Saturation factor: 0.2 (20% haircut)
13 #   - Project lifetime: 50 years
14 #
15 # Calculation:

```

```

16 # Effective capacity relief = 0.7 * 500 = 350 MW
17 #
18 # Curtailment:
19 #     E_curt = 1500 * min(350, 200) = 300,000 MWh/year
20 #     Benefit = 300,000 * $30 = $9M/year
21 #
22 # Congestion (after curtailment allocation):
23 #     Overlap theta = min(1.0, 1500/2000) = 0.75
24 #     H_bc = 0.75 * 2000 = 1500 hours
25 #     H_bnon = 0.25 * 2000 = 500 hours
26 #     DC_remain = 350 - 200 = 150 MW
27 #     E_cong_bc = 1500 * min(150, 300) = 225,000 MWh/year
28 #     E_cong_non = 500 * min(350, 300) = 150,000 MWh/year
29 #     E_cong = 375,000 MWh/year
30 #     Benefit_raw = 375,000 * $50 = $18.75M/year
31 #     Benefit_haircut = (1 - 0.2) * $18.75M = $15M/year
32 #
33 # Residual exceedance (congestion contribution here; curtailment-
only: E_res,cong=0,
34 #     E_res,curt = H_c * max(0, C_c - Delta_C_eff)):
35 #     E_residual = 1500 * max(0, 300-150) + 500 * max(0, 300-350)
36 #                 = 225,000 MWh/year
37 #     Cost = 225,000 * $50 = $11.25M/year
38 #
39 # Output:
40 # - Annual curtailment benefit: $9M
41 # - Annual congestion benefit (haircut): $15M
42 # - Annual residual exceedance cost: $11.25M
43 # - Net annual benefit: $12.75M

```

Integration Example:

```

1 # When called from ctcc.py (main() builds results from
   CongestionReductionResults):
2 csv_manager = CTCCOutputManager()
3 results = {
4     "congestion_benefit_annual": congestion_results.
        annual_congestion_reduction_cost_raw,
5     "congestion_benefit_nominal": congestion_results.
        lifetime_congestion_reduction_cost,
6     "congestion_benefit_pv": congestion_results.
        lifetime_congestion_reduction_cost_pv,
7     "curtailment_benefit_annual": congestion_results.
        annual_curtailment_benefit,
8     "curtailment_benefit_pv": congestion_results.
        lifetime_curtailment_benefit_pv,
9     "residual_exceedance_annual": congestion_results.
        annual_residual_exceedance_cost,

```

```

10     "residual_exceedance_pv": congestion_results.
        lifetime_residual_exceedance_cost_pv,
11     # ... delay costs, haircut variants, physical metrics
12 }
13 csv_manager.add_congestion_curtailement(results)
14 csv_manager.write_batch_summary()

```

1.5 Key Implementation Details

1.5.1 Double-Counting Avoidance

The script prevents double-counting by allocating capacity relief first to curtailment, then to congestion. The overlap factor (θ) accounts for hours when both curtailment and congestion occur. This same allocation is used for delay-period opportunity costs.

1.5.2 Saturation Factors

Saturation factors (haircuts) account for diminishing returns as more capacity is added. They reduce the monetized benefits to reflect that not all theoretical benefits are realized.

1.5.3 Residual Exceedance

The script calculates residual exceedance costs, representing unmet capacity when effective relief is less than the constraint (exceedance or curtailment). Residual exceedance can come from (1) congestion binding hours, (2) curtailment hours, or both. One \$/MWh price is applied (default: average congestion price; the user can set `residual_exceedance_value` when both apply). This is important for partial solutions (e.g., reconductoring or a line that does not fully relieve curtailment). In CTCC, residual exceedance is a system/societal cost categorized under energy/emissions (not under the utility or ratepayer perspective).

1.5.4 Flow Factor

For greenfield projects, the flow factor (ϕ) represents the fraction of nameplate capacity that effectively relieves congestion, accounting for network constraints and power flow physics.

1.5.5 Near-Binding Hours

The script includes near-binding hours with a relief factor to capture benefits from reducing near-constraint conditions, not just fully binding constraints.

1.5.6 Delay and Construction Period Costs

The script calculates delay-period opportunity costs using the same curtailment-first allocation as operations, so overlapping hours are not double counted. This represents the value of forgone

relief while the project is delayed. Present values use `financial_utils.calculate_present_value` with `start_year=calculate_cod_year(delay_years, construction_years)` for benefits (annuity starts at COD); delay/construction costs use `start_year=1` over `delay_years + construction_years`.